



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

Dipartimento di Scienze Fisiche,
Informatiche e Matematiche

4. LAB 2: Introduzione ai *passi* di trasformazione LLVM

Compilatori – Middle end [I215-014]

Corso di Laurea in INFORMATICA
(D.M.270/04) [16-215]
Anno accademico 2025/2026

Prof. Andrea Marongiu
andrea.marongiu@unimore.it

Copyright note

È vietata la copia e la riproduzione dei contenuti e immagini in qualsiasi forma.

È inoltre vietata la redistribuzione e la pubblicazione dei contenuti e immagini non autorizzata espressamente dall'autore o dall'Università di Modena e Reggio Emilia.

Credits

- Pekhimenko, University of Toronto, “Compiler Optimization”

Manipolazione delle istruzioni

Public Member Functions

	Instruction (const Instruction &)=delete
Instruction &	operator= (const Instruction &)=delete
Instruction *	user_back () Specialize the methods defined in Value , as we know that an instruction can only be used by other
const Instruction *	user_back () const
const BasicBlock *	getParent () const
BasicBlock *	getParent ()
const Module *	getModule () const Return the module owning the function this instruction belongs to or nullptr if the function does not l
Module *	getModule ()
const Function *	getFunction () const Return the function this instruction belongs to.
Function *	getFunction ()
void	removeFromParent () This method unlinks 'this' from the containing basic block, but does not delete it.
SymbolTableList< Instruction >::iterator	eraseFromParent () This method unlinks 'this' from the containing basic block and deletes it.
void	insertBefore (Instruction * InsertPos) Insert an unlinked instruction into a basic block immediately before the specified instruction.
void	insertAfter (Instruction * InsertPos) Insert an unlinked instruction into a basic block immediately after the specified instruction.
SymbolTableList< Instruction >::iterator	insertInto (BasicBlock * ParentBB , SymbolTableList< Instruction >::iterator I) Inserts an unlinked instruction into ParentBB at position I and returns the iterator of the inserted i
void	moveBefore (Instruction * MovePos) Unlink this instruction from its current basic block and insert it into the basic block that MovePos liv
void	moveBefore (BasicBlock & BB , SymbolTableList< Instruction >::iterator I) Unlink this instruction and insert into BB before I .
void	moveAfter (Instruction * MovePos) Unlink this instruction from its current basic block and insert it into the basic block that MovePos liv
bool	comesBefore (const Instruction * Other) const Given an instruction Other in the same basic block as this instruction, return true if this instruction c

- Ci sono molti modi (i.e., APIs) per manipolare le istruzioni
 - Instruction, BasicBlock,...
- Come sempre la documentazione ci aiuta a capire quali metodi (APIs) sono disponibili per una data classe

<https://llvm.org/doxygen/index.html>

User – Use - Value

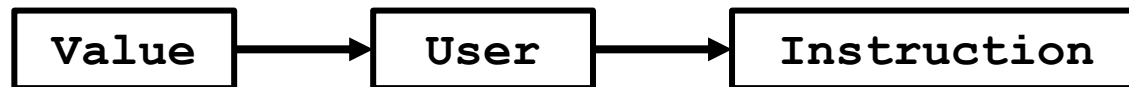
- Supponiamo di dover ottimizzare il seguente codice:

```
%2 = add %1, 0           ; Identità algebrica  
%3 = mul %2, 2
```

- È evidente che la prima istruzione contiene una *identità algebrica* e posso farne a meno
 - Ma cosa succede se semplicemente rimuovo l'istruzione?
 - Il programma va in *crash*, perché **non ho aggiornato le *references* correttamente**
- Come possiamo assicurarci che tutte le *references* (cioè gli *usi*) siano aggiornate correttamente?
- Sfruttando le relazioni ***User - Use - Value*** di LLVM

User – Use - Value

- Le istruzioni (`Instruction`) LLVM ereditano dalla classe `Value` (come quasi tutte le classi LLVM)
- Ma ereditano anche dalla classe `User`
 - Quindi esiste implicitamente un legame tra le **istruzioni** e i loro **usi**
- In altre parole, le `Instruction` giocano entrambi i ruoli di **User** e **Usee** (`Value`)



GERARCHIA DELLE CLASSI LLVM

Value

- La classe `Value` è la più importante classe base in LLVM, dato che quasi tutti i tipi di oggetto ereditano da questa
- Un nodo `Value` ha un *tipo* (es., integer, floating point): **`getType()`**
- Un nodo `Value` può avere o meno un *nome*: **`hasName()`, `getName()`**
- Soprattutto, un nodo `Value` ha una lista di *Users* che lo utilizzano

Istruzioni come User

Un oggetto `Instruction` è anche un oggetto `User`

- Ogni `User (Instruction)` ha una lista di valori che sta utilizzando. Questi valori sono gli **operandi** dell'istruzione, e sono oggetti di tipo `Value`.

```
User &Inst = ...  
for (auto Iter = Inst.op_begin(); Iter != Inst.op_end(); ++Iter)  
    { Value *Operand = *Iter; }
```

Se eseguo questo codice per analizzare l'istruzione

`%2 = add %1, 0`

verranno estratti gli operandi

`%1, 0`

Istruzioni come Usee

Perché un oggetto `Instruction` è anche uno **Usee**?

- La risposta risiede nel come dobbiamo interpretare un oggetto `Instruction` LLVM:

```
%2 = add %1, 0
```

-  Il risultato dell'istruzione `add %, 0` viene assegnato a `%2`
-  `%2` è la rappresentazione `Value` dell'istruzione `add %1, 0`

- Quindi, ogni volta che nel testo usiamo il valore `%2` in realtà intendiamo proprio indicare l'istruzione `add %1, 0`

Ricapitolando...

Torniamo a considerare l'esempio di prima

```
%2 = add %1, 0           ; Identità algebrica  
%3 = mul %2, 2
```

Sia `Inst` una reference alla prima istruzione:

Da `User` l'istruzione usa degli operandi

```
for (auto Iter = Inst.op_begin(); Iter != Inst.op_end(); ++Iter)  
    { Value *Operand = *Iter; }
```

→ **Operand %1, 0**

Ma ha a sua volta degli users (ovvero, le istruzioni di cui è una usee)

```
for (auto Iter = Inst.user_begin(); Iter != Inst.user_end(); ++Iter)  
    { User *InstUser = *Iter; }
```

→ **Instruction mul %2, 2 (oppure Value %3)**

Vediamo se abbiamo capito

Ipotizziamo di avere il seguente codice C

```
y = p + 1;  
y = q * 2;  
z = y + 3;
```

Cosa dovrebbe essere ritornato dalla routine che ispeziona gli *users* dell'istruzione $y = p + 1$?

Vediamo se abbiamo capito

Ipotizziamo di avere il seguente codice C

```
y = p + 1;  
y = q * 2;  
z = y + 3;
```

Cosa dovrebbe essere ritornato dalla routine che ispeziona gli *users* dell'istruzione $y = p + 1$?

- **Niente!**



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

Dipartimento di Scienze Fisiche,
Informatiche e Matematiche

LAB 2 – TRANSFORM PASS (LocalOpts)

LocalOpts – Sorgente

- Scaricate il sorgente per il Lab 2

`LocalOpts.cpp`

solo codice del passo, da integrare in uno scheletro di Function pass analogo a quello di TestPass

`Foo.ll`

programma di test

- Create quindi la seguente struttura di cartelle per il LAB2, e copiateci dentro i file scaricati come indicato

LAB 2

|__ *BUILD/*

|__ *TEST/*

|__ *Foo.ll*

|__ *SRC/*

|__ *LocalOpts.cpp*

|__ *CMakeLists.txt*



Vedi prossima slide

LocalOpts – Sorgente

- Modificare quindi i seguenti file:
 - LocalOpts.cpp
 - Dovete ricostruire la struttura del passo di Testpass, inclusa la parte di registrazione del plugin. Il metodo `run` prende in ingresso un handle ad un oggetto `Function`. Potete passare lo stesso handle al metodo `runOnFunction` trovato nel *LocalOpts.cpp* originale.
 - Rinominate anche il flag di attivazione del passo da *test-pass* in *local-opts*
 - CMakeLists.txt
 - Anche in questo caso ripartite dal CMakeLists.txt fornito per TestPass. Modificate la sezione 3. ADD THE TARGET perché compili il file LocalOpts come modificato

Compilazione del plugin

- Ora siamo pronti a compilare il nuovo plugin
- Assicuriamoci di avere nel PATH la cartella all'installazione di LLVM19

```
export PATH=<your/path/to/LLVM19>/INSTALL/bin:$PATH
```

- Assumendo di trovarci nella cartella Lab 2, spostiamoci nella cartella BUILD, quindi avviamo la configurazione

```
cd BUILD  
cmake -DLT_LLVM_INSTALL_DIR  
=<your/path/to/LLVM19>/INSTALL/bin ../SRC
```

- A questo punto il makefile è generato e possiamo compilare il plugin

```
make
```


Esecuzione del passo

- Possiamo quindi finalmente invocare il passo. Spostiamoci nella cartella TEST, dove è presente Foo.ll

```
cd ../TEST
```

- Ricordiamoci di agganciare il plugin a `opt` con `-load-pass-plugin`

```
opt -S -load-pass-plugin ../BUILD/libLocalOpts.so -p  
local-opts \ Foo.ll -o Foo.optimized.ll
```

- Se non usiamo il flag `-S` l'output è in formato bytecode, quindi dobbiamo poi usare il disassembler per generare la forma `.ll` leggibile

```
llvm-dis Foo.optimized.bc -o Foo.optimized.ll
```

Esercizio 1 – Studio del passo

- Confrontare `Foo.ll` e `Foo.optimized.ll`, e tramite lo studio del passo in `LocalOpts.cpp` capire cosa fa (e come lo fa)
- Familiarizzare un po' con le varie primitive di manipolazione della IR proposte nel passo
- Studiare la documentazione, rispondere alla domanda presente nel commento verso la fine del passo
`LocalOpts.cpp`

Esercizio 2 – Un passo più utile

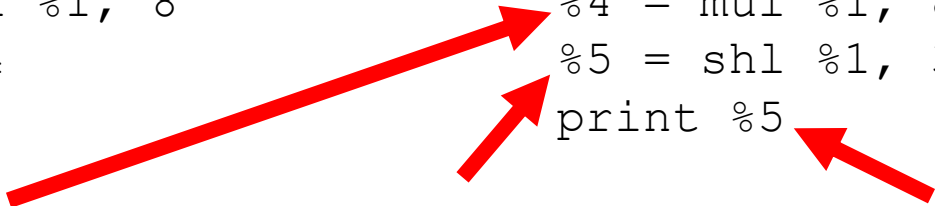
- Per esercitarci a manipolare la IR modifichiamo adesso il passo LocalOpts.cpp perché sostituisca tutte le operazioni di **moltiplicazione** che hanno tra gli operandi una costante che è una potenza di 2 con una **shift** (*strength reduction*)

Es.

```
%1 = add %2, %3  
%4 = mul %1, 8  
print %4
```

DIVENTA

```
%1 = add %2, %3  
%4 = mul %1, 8  
%5 = shl %1, 3  
print %5
```



NOTA: Non rimuoviamo la **mul**, ma la slinkiamo dal programma rimpiazzando i suoi usi con gli usi della nuova istruzione. A seguire applichiamo un passo di DCE (dead code elimination) per rimuoverla (`opt -passes=dce ...`)